

# Requintu

## Documentación Técnica del Proyecto

Plataforma web + app Android de negocios locales de Colombia

Backend: Node.js / Express — Frontend: React / Vite

Base de datos: MySQL (TiDB Cloud) — Imágenes: Cloudinary

Repositorio: [github.com/aycardy93-eng/-requintu](https://github.com/aycardy93-eng/-requintu)

Versión 1.0 — 28 de agosto de 2026

# 1. Resumen del proyecto

---

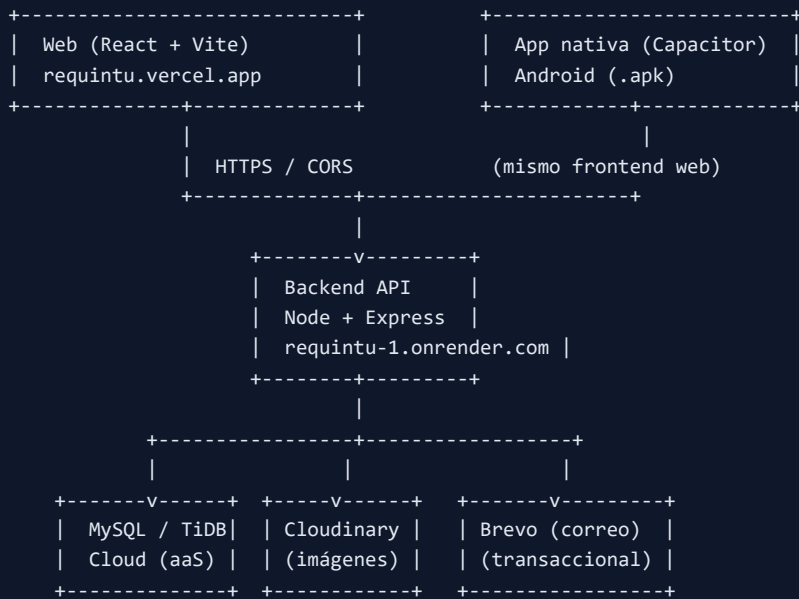
**Requintu** es una aplicación que conecta a viajeros y usuarios con los negocios locales de Colombia. Los usuarios pueden explorar locales a través de un **mapa interactivo** dividido por departamentos y municipios, consultar el detalle de cada negocio (fotos, promociones, calificaciones, ubicación), publicar contenido en un muro comunitario y, en el caso de los comerciantes, crear y administrar sus propios locales. Una cuenta de administrador permite gestionar usuarios, locales y publicaciones en un panel dedicado.

El proyecto está formado por dos clientes y un solo backend:

- **Web** (SPA React) desplegada en Vercel.
- **App nativa Android** (Capacitor 8) que empaqueta el mismo frontend.
- **API REST** en Node.js/Express desplegada en Render.

## 2. Arquitectura general

El sistema sigue una arquitectura de **cliente-servidor** con API REST y autenticación por tokens JWT:



- **Frontend:** React 19 con React Router 7 (SPA). Compilado con Vite 8; se sirve como estático en Vercel con rewrites SPA.
- **App nativa:** Capacitor 8 genera el proyecto Android en frontend/android; el WebView carga el mismo dist.
- **Backend:** Express 4 (módulos ES). Expone `/api/*`, sirve `/uploads` y aplica seguridad (helmet, rate limit, CORS, HTTPS).
- **Base de datos:** MySQL 8 administrada (TiDB Cloud) vía `mysql2` (pool de conexiones, TLS).
- **Almacenamiento:** las imágenes se suben al backend con Multer y se publican en Cloudinary.
- **Correo:** la recuperación de contraseña usa la API v3 de Brevo (SMTP por HTTP).

### 3. Stack tecnológico

#### 3.1 Backend (raíz del repositorio)

| Tecnología              | Uso                                                                          |
|-------------------------|------------------------------------------------------------------------------|
| Node.js (ESM)           | Entorno de ejecución. <code>npm start</code> ejecuta <code>index.js</code> . |
| Express 4               | Servidor HTTP y enrutado REST.                                               |
| mysql2                  | Pool de conexiones a MySQL / TiDB Cloud.                                     |
| jsonwebtoken + bcryptjs | Access tokens JWT (1 h) y hash de contraseñas (bcrypt, 10 rondas).           |
| cookie-parser           | Lectura de la cookie <code>httpOnly</code> de refresh token.                 |
| multer                  | Recepción de archivos (solo imágenes JPEG/PNG/WebP, máx. 5 MB).              |
| cloudinary (SDK 2)      | Publicación de imágenes en la nube.                                          |
| express-validator       | Validación y saneamiento de entradas.                                        |
| express-rate-limit      | Límite de peticiones por IP (API y autenticación).                           |
| helmet                  | Cabeceras de seguridad HTTP (HSTS, CORS cross-origin para recursos).         |
| cors                    | Lista blanca de orígenes (web + Capacitor).                                  |
| dotenv                  | Variables de entorno.                                                        |

#### 3.2 Frontend (carpeta frontend)

| Tecnología                            | Uso                                                     |
|---------------------------------------|---------------------------------------------------------|
| React 19 + Vite 8                     | SPA y empaquetado de producción.                        |
| react-router-dom 7                    | Enrutado del cliente.                                   |
| Capacitor 8 (core, cli, android, app) | Empaquetado nativo Android y control del botón «atrás». |
| swiper                                | Carrusel de imágenes (login e inicio).                  |
| oxlint                                | Linter ( <code>npm run lint</code> ).                   |
| sharp                                 | Dev: generación de iconos de la app.                    |

## 4. Estructura del repositorio

```
proyecto/
├─ index.js           # API principal (Express, ~1300 líneas)
├─ db.js              # Pool de conexiones MySQL + health check
├─ filtroProfano.js   # Filtro de groserías (leetspeak, repetidos)
├─ .env.example       # Plantilla de variables de entorno
├─ requintu_dump.sql  # Respaldo del esquema/datos
├─ vercel.json        # Config de despliegue del frontend (SPA)
├─ netlify.toml       # Alternativa de despliegue (Netlify)
├─ package.json       # Dependencias del backend
├─ frontend/
│   ├─ capacitor.config.json # Config de Capacitor (appId, webDir)
│   ├─ index.html         # HTML base (sin service worker)
│   └─ vite.config.js
│       └─ src/
│           ├─ main.jsx      # Punto de entrada (React Router + AuthProvider)
│           ├─ App.jsx       # Rutas de la SPA
│           ├─ config.js     # API_URL según VITE_BACKEND_ORIGIN
│           ├─ context/AuthContext.jsx # Sesión: login, logout, refresco automático
│           ├─ components/   # Navbar, ProtectedRoute, AdminRoute, InstallBanner, FondoPagina
│           ├─ pages/        # Home, Login, Register, MapaColombia, Locales,
│           │               # LocalDetalle, CrearLocal, Publicaciones, Perfil,
│           │               # Admin, OlvidePassword, ResetPassword
│           └─ assets/       # co-departamentos.svg, imágenes
```

## 5. Backend

### 5.1 Punto de entrada (`index.js`)

Configura Express, aplica los middlewares globales y registra más de **30 rutas** bajo `/api`. En producción fuerza HTTPS (redirect 301), confía en el proxy (trust proxy), activa HSTS y usa cookies `SameSite=None; Secure` para permitir el login desde la app nativa.

### 5.2 Conexión a la base de datos (`db.js`)

Crea un pool con `mysql2/promise` (límite de 5 conexiones por defecto, keep-alive). Si `DB_SSL=true` (exigido por TiDB Cloud) activa TLS. Exporta `checkDatabaseHealth()` que valida la conexión con `SELECT 1`.

### 5.3 Filtro de groserías (`filtroProfano.js`)

Normaliza el texto (minúsculas, leetspeak `0-o, 3-e, 4-a, @-a...`, caracteres repetidos: `joder` → `joder`) y lo compara contra una lista en español. Evita falsos positivos en palabras cortas (longitud mínima). Se aplica con el middleware `sinGroserias(campos)` a locales, planes, calificaciones y publicaciones.

### 5.4 Autenticación y autorización

- **Registro:** valida nombre, correo (único) y contraseña (mín. 8). Roles permitidos: *turista* (por defecto) y *comerciante*. Contraseña cifrada con `bcrypt`.
- **Login:** devuelve un **access token JWT (1 h)** con `{ id, nombre, rol }` y emite una **cookie httpOnly** con el refresh token.
- **Refresh token:** generado con `crypto.randomBytes(64)`, se guarda *hasheado* (SHA-256) en la tabla `refresh_tokens` con expiración de 7 días. Cada refresco **rota** el token (se revoca el anterior). La cookie se limita a la ruta `/api/auth`.
- **Logout:** revoca el refresh token y limpia la cookie.
- **Protección:** `authMiddleware` verifica el JWT (cabecera `Authorization: Bearer`) y rellena `req.user; checkRole([...])` restringe acciones a roles concretos (p. ej. crear locales requiere *comerciante* o *admin*).

### 5.5 Recuperación de contraseña (Brevo)

`/api/forgot-password` genera un token (64 hex, expira en 1 h) guardado *hasheado* en `password_resets` y envía un correo con el enlace mediante la **API v3 de Brevo** (fetch HTTPS a `api.brevo.com/v3/smtp/email`). `/api/reset-password` valida el token (un solo uso), actualiza la contraseña, marca el reset como usado y revoca las sesiones previas.

### 5.6 Seguridad

| Medida                | Implementación                                                                      |
|-----------------------|-------------------------------------------------------------------------------------|
| Cabeceras HTTP        | helmet + HSTS (365 días) + redirect obligatorio a HTTPS en producción.              |
| CORS                  | Lista blanca (local, Vercel y Capacitor: <code>capacitor://localhost</code> ).      |
| Rate limiting         | 300 req / 15 min en <code>/api</code> ; 10 req / 15 min en login y registro.        |
| Validación de entrada | <code>express-validator</code> en todas las rutas de escritura.                     |
| Contenido inapropiado | Middleware <code>sinGroserias</code> .                                              |
| Subida de archivos    | Solo <code>image/jpeg png webp</code> , extensión y MIME comprobados, máx. 5 MB.    |
| Contraseñas           | <code>bcrypt</code> (10 rondas); nunca en texto plano.                              |
| Tokens                | JWT firmado con <code>JWT_SECRET</code> ; refresh tokens hasheados en BD y rotados. |

### 5.7 Subida de imágenes

POST `/api/upload` (protegido) recibe la imagen con Multer, la sube a **Cloudinary** mediante el SDK `uploader.upload` (carpeta según entidad) y devuelve la URL pública. El frontend usa esta URL para persistir locales, planes y publicaciones.

## 6. API REST (resumen de endpoints)

### 6.1 Autenticación y perfil

| Método | Ruta                 | Acceso      | Descripción                                |
|--------|----------------------|-------------|--------------------------------------------|
| POST   | /api/register        | Público     | Registro (turista/comerciante).            |
| POST   | /api/login           | Público     | Inicio de sesión (JWT + cookie refresh).   |
| POST   | /api/auth/refresh    | Cookie      | Renueva access token (rota refresh).       |
| POST   | /api/auth/logout     | Cookie      | Cierra sesión.                             |
| POST   | /api/forgot-password | Público     | Envía enlace de recuperación (Brevo).      |
| POST   | /api/reset-password  | Público     | Reestablece contraseña con token de 1 uso. |
| GET    | /api/perfil          | Autenticado | Datos del perfil.                          |

### 6.2 Catálogo

| Método | Ruta                                  | Acceso  | Descripción                           |
|--------|---------------------------------------|---------|---------------------------------------|
| GET    | /api/municipios                       | Público | Lista de municipios.                  |
| GET    | /api/categorias                       | Público | Categorías de locales.                |
| GET    | /api/departamentos                    | Público | Departamentos con datos para el mapa. |
| GET    | /api/departamentos/:nombre/municipios | Público | Municipios de un departamento.        |

### 6.3 Locales, calificaciones y planes

| Método       | Ruta                            | Acceso            | Descripción                                    |
|--------------|---------------------------------|-------------------|------------------------------------------------|
| GET          | /api/locales                    | Público           | Listado con filtros y búsqueda.                |
| POST         | /api/locales                    | Comerciante/Admin | Crea un local (valida unicidad por municipio). |
| GET          | /api/locales/:id                | Público           | Detalle con calificación promedio.             |
| PUT          | /api/locales/:id                | Autenticado       | Edita (dueño/admin; evita local repetido).     |
| DELETE       | /api/locales/:id                | Autenticado       | Elimina (dueño/admin).                         |
| GET          | /api/locales/:id/calificaciones | Público           | Reseñas del local.                             |
| POST         | /api/locales/:id/calificaciones | Autenticado       | Puntúa (1-5) y comenta.                        |
| GET          | /api/locales/:id/planes         | Público           | Planes/paquetes del local.                     |
| POST         | /api/locales/:id/planes         | Autenticado       | Agrega plan.                                   |
| PUT / DELETE | /api/planes/:id                 | Autenticado       | Edita / elimina un plan.                       |

### 6.4 Subida y publicaciones

| Método       | Ruta                   | Acceso        | Descripción                    |
|--------------|------------------------|---------------|--------------------------------|
| POST         | /api/upload            | Autenticado   | Sube imagen a Cloudinary.      |
| GET          | /api/publicaciones     | Público       | Muro de publicaciones.         |
| POST         | /api/publicaciones     | Turista/Admin | Publica contenido.             |
| PUT / DELETE | /api/publicaciones/:id | Autenticado   | Edita / elimina (autor/admin). |

### 6.5 Administración (solo rol admin)

| Método | Ruta | Descripción |
|--------|------|-------------|
|--------|------|-------------|

|        |                              |                                    |
|--------|------------------------------|------------------------------------|
| GET    | /api/admin/estadisticas      | Totales y distribución por rol.    |
| GET    | /api/admin/usuarios          | Listado con paginación y rol.      |
| PUT    | /api/admin/usuarios/:id/rol  | Cambia el rol de un usuario.       |
| DELETE | /api/admin/usuarios/:id      | Elimina un usuario.                |
| GET    | /api/admin/locales           | Listado paginado de locales.       |
| GET    | /api/admin/publicaciones     | Listado paginado de publicaciones. |
| DELETE | /api/admin/publicaciones/:id | Modera (elimina) una publicación.  |



## 7. Base de datos

MySQL 8 (TiDB Cloud). Esquema principal exportado en requintu\_dump.sql:

| Tabla           | Contenido                                                                                                   | Claves principales          |
|-----------------|-------------------------------------------------------------------------------------------------------------|-----------------------------|
| usuarios        | Nombre, email único, password_hash, rol (admin/comerciante/turista), foto_perfil, reset_token y expiración. | PK id_usuario; UNIQUE email |
| municipios      | ~1120 municipios de Colombia con su departamento.                                                           | PK id_municipio             |
| categorias      | 13 categorías (restaurante, café, alojamiento, turismo, artesanías, etc.).                                  | PK id_categoria             |
| locales         | Nombre, descripción, dirección, teléfono, imagen, FK a categoría, municipio y propietario.                  | FKs con ON DELETE SET NULL  |
| calificaciones  | Reseñas: puntuación (1-5 con CHECK), comentario, fecha; usuario y local.                                    | FKs con ON DELETE CASCADE   |
| planes          | Planes/paquetes del local: título, descripción, precio, fechas, imagen.                                     | FK id_local                 |
| publicaciones   | Contenido del muro, imagen opcional, fecha, autor.                                                          | FK usuario_id               |
| refresh_tokens  | Refresh tokens (SHA-256), expiración, revocado, creado_en.                                                  | UNIQUE token_hash           |
| password_resets | Tokens de recuperación (hash, expiración, usado). Se crea en producción por las rutas de contraseña.        | FK id_usuario               |
| posts           | Tabla heredada de una iteración anterior (sin uso activo en la UI).                                         | FK id_usuario               |

*Nota: en los INSERT del respaldo los acentos aparecen con caracteres de escape (<QuindÃ–O>); las tablas usan juego de caracteres utf8mb4.*

## 8. Frontend

### 8.1 Inicio y rutas (main.jsx, App.jsx)

main.jsx monta React 19 con `BrowserRouter` y `AuthProvider`. App.jsx define las rutas públicas ( `/`, `/login`, `/register`, `/olvide-password`, `/reset-password` ), las de lectura ( `/publicaciones`, `/locales`, `/locales/:id` ) y las protegidas ( `/mapa`, `/crear-local`, `/perfil` con `ProtectedRoute` ; `/admin` con `AdminRoute` ). También registra un listener del **botón atrás** de Android vía Capacitor (vuelve atrás o cierra la app).

### 8.2 Gestión de sesión (context/AuthContext.jsx)

- El **access token vive en memoria** (estado React); **nunca** se guarda en localStorage ni en cookies legibles por JavaScript.
- Al montar la app se llama `/api/auth/refresh` con la cookie `httpOnly` para restaurar la sesión.
- Un temporizador renueva el token **5 minutos antes** de la expiración (exp del JWT), evitando sesiones caídas.
- **Logout** limpia el estado y revoca el token en el servidor.
- El rol (admin/comerciante/turista) se decodifica del JWT y condiciona menús y rutas.

### 8.3 Pantallas principales

| Página                         | Función                                                                                                                                                                                                                             |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Home                           | Portada con carrusel (swiper) y accesos a secciones.                                                                                                                                                                                |
| Login / Register               | Acceso con carrusel decorativo y diseño responsive (Login.css).                                                                                                                                                                     |
| OlvidePassword / ResetPassword | Flujo de recuperación de contraseña.                                                                                                                                                                                                |
| MapaColombia                   | Mapa SVG de departamentos (assets/co-departamentos.svg) con zoom/pan propio (Botones +/-, arrastre, doble toque). Al tocar un departamento se muestran sus municipios en un panel flotante y se navega a los locales del municipio. |
| Locales                        | Filtros por departamento, municipio y categoría; tarjetas con estrellas y promociones.                                                                                                                                              |
| LocalDetalle                   | Fotos, descripción, mapa de Google, planes/promociones, galería de calificaciones y edición (dueño).                                                                                                                                |
| CrearLocal                     | Formulario + vista previa del mapa; alta de local.                                                                                                                                                                                  |
| Publicaciones                  | Muro de publicaciones con imágenes y autor.                                                                                                                                                                                         |
| Perfil                         | Datos personales y edición.                                                                                                                                                                                                         |
| Admin                          | Panel: resumen de estadísticas y pestañas Usuarios/Locales/Publicaciones con paginación. Las tablas son adaptables (scroll horizontal en móvil).                                                                                    |

### 8.4 Configuración de entorno

config.js lee `VITE_BACKEND_ORIGIN` (variable de compilación) y construye `API_URL`; si no está definida usa `http://localhost:3000`. En la app nativa se compila apuntando a `https://requintu-1.onrender.com`.

## 9. App nativa Android (Capacitor)

---

- **Configuración:** frontend/capacitor.config.json con appld `com.requintu.app`, appName Requintu y webDir `dist`.
- **Proyecto nativo:** generado con `npx cap add android`; el web build se copia con `npx cap sync android`.
- **Compilación:** Gradle 8.14.3 (JDK 21), compileSdk/targetSdk 36, minSdk 24. Genera app-debug.apk (firmado en debug, instalable).
- **Marca:** iconos propios en todas las densidades y fondo #12283D.
- **Interacción nativa:** botón «atrás» gestionado con @capacitor/app.
- **Backend compatible:** CORS admite `capacitor://localhost` y las cookies en producción usan `SameSite=None` para funcionar dentro del WebView.

## 10. Despliegue

---

### 10.1 Backend (Render)

- URLs: `https://requintu-1.onrender.com` (principal) y `https://requintu.onrender.com`.
- Comando de inicio: `npm start`. Requiere `NODE_ENV=production` y las variables de entorno según `.env.example`.
- El proceso valida `JWT_SECRET` al arrancar y verifica la conexión a la BD.

### 10.2 Frontend (Vercel)

- URL: `https://requintu.vercel.app`. Se despliega desde la rama `main` de GitHub (push automático).
- `vercel.json`: framework Vite, build `npm run build`, `outputDirectory`: `dist` y **rewrites SPA** `/(.*) → /index.html`.
- Variable de compilación: `VITE_BACKEND_ORIGIN=https://requintu-1.onrender.com`.
- Nota: la PWA (service worker) se desactivó a propósito para que la web cargue siempre la versión más reciente.

### 10.3 Netlify (alternativa)

`netlify.toml` define el build en frontend y un redirect SPA `/* → /index.html`.

### 10.4 Base de datos (TiDB Cloud)

MySQL en la nube, host `gateway01.us-east-1.prod.aws.tidbcloud.com`, puerto 4000, con SSL configurado `rejectUnauthorized: true`.

## 11. Variables de entorno

---

```
NODE_ENV=production|development
JWT_SECRET=<secreto largo>
DB_HOST, DB_USER, DB_PASSWORD, DB_NAME, DB_SSL
CORS_ORIGINS=<orígenes extra separados por coma>
RATE_LIMIT_API_MAX, RATE_LIMIT_AUTH_MAX
BREVO_API_KEY, BREVO_SENDER_EMAIL
APP_URL=<URL pública del frontend>
PORT
CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET
VITE_BACKEND_ORIGIN=<frontend, en compilación>
```

*Advertencia: el archivo .env real no debe subirse a git (está ignorado).*

## 12. Ejecución local

---

### 12.1 Backend

```
npm install
cp .env.example .env # completar valores (BD, JWT, Cloudinary, Brevo)
npm run dev          # o npm start
```

### 12.2 Frontend

```
cd frontend
npm install
npm run dev          # Vite (puerto 5173)
npm run lint         # oxlint
npm run build        # compilación de producción
```

### 12.3 App Android

```
cd frontend
VITE_BACKEND_ORIGIN=https://requintu-1.onrender.com npm run build
npx cap sync android
cd android
JAVA_HOME=<jdk 21> ANDROID_HOME=<sdk> ./gradlew assembleDebug
# APK en android/app/build/outputs/apk/debug/app-debug.apk
```

## 13. Seguridad y buenas prácticas aplicadas

---

- Contraseñas con bcrypt y tokens JWT con expiración corta, + refresh tokens rotatorios en cookie httpOnly (mitigan robo y reuso).
- Cabeceras de seguridad (helmet/HSTS), HTTPS forzado y CORS por lista blanca.
- Rate limiting por IP en API y autenticación.
- Validación de entradas y saneamiento en todas las rutas de escritura.
- Filtro de lenguaje inapropiado para contenido generado por usuarios.
- Restricción de subidas de archivos por formato y tamaño.
- Variables sensibles únicamente en variables de entorno (nunca en el repositorio).
- Verificación de unicidad del nombre de un local dentro de su municipio.

## 14. Mejoras futuras sugeridas

---

- Firma **release** del APK y publicación en Google Play.
- Pruebas automatizadas en el backend (script `npm test` definido) y test e2e del frontend.
- WebSockets o notificaciones push para avisos del muro.
- Moderación manual de locales antes de publicarlos (flujo de aprobación).
- Reintentos y cola de envío de correos (Brevo) para mayor robustez.
- Migraciones versionadas del esquema (hoy se respalda con dump).



## 15. Glosario de rutas del servidor

---

- `/api/*` — API REST.
- `/uploads/*` — archivos estáticos locales (legacy de imágenes).
- Cualquier otra ruta en web la resuelve el SPA (rewrites de Vercel/Netlify).

Fin de la documentación técnica.